CATEGORY B

Coding Assignment

Data Structures · Arrays, Hashing, Two Pointers, Sliding Window, Binary Search

Problem 1: Class Topper Finder

TASK

A teacher stores the marks of a class as a 2D grid, where each row belongs to one student and each column is one subject. The teacher wants to know which student has the highest total marks.

Implement a function `findTopper(marks)` that returns a tuple `(rowIndex, total)` for the student with the highest total. If two or more students have the same highest total, return the one with the smallest row index.

INPUT / OUTPUT

Input: `marks` — a list of lists of non-negative integers, e.g., `[[78, 85, 90], [88, 92, 79]]`. Output: A tuple `(rowIndex, total)`.

CONSTRAINTS

- 1 <= number of students <= 500
- 1 <= number of subjects <= 10
- 0 <= marks[i][j] <= 100

EXAMPLE

```
marks = [[78, 85, 90],
         [88, 92, 79],
         [65, 70, 95]]

Expected Output: (1, 259) // totals are 253, 259, 230
```

COMPLEXITY REQUIREMENTS

State the time complexity in terms of m (students) and n (subjects), and the additional space complexity beyond the input grid.

HINT

Use a nested loop: the outer loop goes through each student, and the inner loop adds up that student's marks. Update the best answer only when a total is strictly greater than the current best.

Problem 2: Merging Two Token Queues

TASK

A hospital runs two registration counters. Each counter produces a list of token numbers already sorted in ascending order. At the end of the day, the two lists must be combined into one list, still sorted in ascending order, for the daily report.

Implement a function `mergeTokens(counterA, counterB)` that returns a single sorted list containing all tokens from both lists, including any duplicates. Do not combine the lists and then sort them; use the fact that both lists are already sorted.

INPUT / OUTPUT

Input: `counterA`, `counterB` — two lists of integers, each sorted in ascending order (either may be empty).

Output: One list of integers sorted in ascending order.

CONSTRAINTS

- 0 <= length of each list <= 10^5
- Duplicates are allowed and must be kept

EXAMPLES

Sample 1

```
counterA = [3, 8, 15, 20]
counterB = [5, 8, 12]

Expected Output: [3, 5, 8, 8, 12, 15, 20]
```

Sample 2

```
counterA = []
counterB = [4, 9]

Expected Output: [4, 9]
```

COMPLEXITY REQUIREMENTS

State the time and space complexity of your solution in terms of m and n (the lengths of the two lists). Compare it with the approach of joining the lists and sorting the result.

HINT

Use two pointers, one for each list. At every step, copy the smaller of the two current values and move that pointer forward. When one list runs out, copy the rest of the other list.

Problem 3: Most Popular Canteen Order

TASK

A college canteen records every order of the day as a list of item names. The manager wants to know which item was ordered the most and how many times.

Implement a function `mostPopular(orders)` that returns a tuple `(item, count)` for the most frequently ordered item. If two or more items have the same highest count, return the item that appears first in the orders list.

INPUT / OUTPUT

Input: `orders` — a list of strings (item names), e.g., `["dosa", "idli", "dosa"]`. Output: A tuple `(item, count)`.

CONSTRAINTS

- 1 <= length of orders <= 10^5
- Item names are lowercase words

EXAMPLES

Sample 1

```
orders = ["dosa", "idli", "vada", "dosa", "idli", "dosa", "tea"]

Expected Output: ("dosa", 3)
```

Sample 2

```
orders = ["tea", "coffee", "coffee", "tea"]

Expected Output: ("tea", 2)
```

COMPLEXITY REQUIREMENTS

State the time and space complexity of a solution that counts each item by scanning the whole list again, and of your solution that uses a hash-based structure.

HINT

Store a count for each item in a hash map in one pass. Then go through the orders list again in its original order to pick the first item that has the highest count, so ties are broken correctly.

Problem 4: Hot Weather Alert Windows

TASK

A weather station records one temperature reading per hour. An alert is raised for every block of k consecutive hours whose average temperature is greater than or equal to a given threshold.

Implement a function `countAlerts(readings, k, threshold)` that returns the number of k-hour blocks (contiguous subarrays of size k) whose average is at least `threshold`. Blocks may overlap.

INPUT / OUTPUT

Input: `readings` — a list of integers; `k` — the block size; `threshold` — an

integer. Output: An integer: the number of qualifying blocks.

CONSTRAINTS

- 1 <= k <= length of readings <= 10^5
- 0 <= readings[i], threshold <= 10^4

EXAMPLE

```
readings = [2, 2, 2, 2, 5, 5, 5, 8], k = 3, threshold = 4

Expected Output: 3 // blocks [2, 5, 5], [5, 5, 5], [5, 5, 8]
```

COMPLEXITY REQUIREMENTS

State the time complexity of recalculating the sum of every block from scratch, and the time and space complexity of your sliding-window solution.

HINT

Keep the sum of the current block. When the block slides one hour forward, add the new reading and subtract the one that left. Compare sum >= k × threshold to avoid decimal division.

Problem 5: Ticket Price Slot Finder

TASK

An event website keeps its list of ticket prices sorted in ascending order, with no duplicate prices. When the organiser wants to add a new price, the website must find the index at which that price already exists, or the index where it should be inserted so the list stays sorted.

Implement a function `findSlot(prices, newPrice)` that returns this index. Your solution must use the sorted order to avoid checking prices one by one.

INPUT / OUTPUT

Input: `prices` — a sorted list of distinct integers; `newPrice` — an integer.

Output: An integer index between 0 and the length of prices (inclusive).

CONSTRAINTS

- 0 <= length of prices <= 10^5
- prices is sorted in ascending order with no duplicates

EXAMPLES

Sample 1

```
prices = [120, 150, 200, 260], newPrice = 150

Expected Output: 1 // already present at index 1
```

Sample 2

```
prices = [120, 150, 200, 260], newPrice = 210

Expected Output: 3
```

Sample 3

```
prices = [120, 150, 200, 260], newPrice = 300

Expected Output: 4
```

COMPLEXITY REQUIREMENTS

State the time and additional space complexity of a linear scan and of your binary-search solution.

HINT

Run a normal binary search. If the price is not found, the loop ends with the low pointer sitting exactly at the position where the price belongs.